第 4-1 讲 • 改代码的安全流程

> **模块 4 • 学会改代码** ｜ 建议用时 **40 分钟**（阅读 18 + 动手 22）
> 前置要求：完成模块 3（会读代码、会追数据流）

本节目标

读完并做完这一讲，你能：

1. 说出一套**改代码的标准流程**（六步），并知道每一步在防什么。
2. 理解**"基线"**的概念：为什么动手前必须先确认"现在是好的"。
3. 知道**你的改动会不会碰到账本数据**——以及怎么判断。
4. 认识项目的 **14 个验证命令**，知道每个在防什么问题。
5. 掌握**版本控制（git）**的基本用法，以及没有它时怎么保护自己。

一、为什么"改代码"需要流程

1.1 现实比喻：动手术前的检查单

外科医生动刀前会做一件事：**核对检查单。**

- 确认病人身份（别开错人）
- 确认手术部位（别开错腿）
- 确认过敏史
- 确认器械齐备

为什么：因为手术**不可逆**，一旦切错就没法撤销。检查单花 2 分钟，能避免灾难。

改代码是一样的：

- 改错了，用户的数据可能坏掉（**不可逆**）
- 改错了，可能几个功能一起坏（**连锁反应**）
- 改错了，你自己可能都不知道改坏在哪（**难定位**）

所以改代码也需要一份"检查单"。

1.2 六步流程（本讲的核心）

```
① 确认基线  →  现在是好的吗？（测试全绿吗）
② 分析影响  →  要改哪几个文件？会波及谁？
③ 备份      →  改坏了能还原吗？
④ 最小改动  →  只改必须改的
⑤ 验证      →  新功能对吗？旧功能还在吗？
⑥ 收尾      →  保留还是还原？需要清理什么？
```

每一步都在防一种具体的失败：

步骤	防的是什么
① 确认基线	把**原本就有的问题**误当成自己改出来的
② 分析影响	**漏改**（改了界面忘了改数据库）
③ 备份	改坏了**无法还原**
④ 最小改动	出问题时**定位不到**是哪处改的
⑤ 验证	**破坏了别的功能**却没发现
⑥ 收尾	测试数据、临时文件**留在项目里**

第 ① 步特别容易被跳过，也特别重要。 后面会专门讲。

二、你的改动会不会碰到账本数据

这是你最该关心的问题，而且答案很明确：

2.1 三种改动，三种风险

改动类型	会不会碰你的账本	例子
只改界面	❌ 完全不会	改文字、颜色、布局、加显示列（3-4 讲那种）
改主进程逻辑	⚠️ 取决于跑了什么	改校验规则会影响新记录，但**不会动已有数据**
数据库迁移	✅ **会。**这是唯一会改已有数据的操作	加字段、改表结构（2-3 讲那种）

结论：**你在学习过程中做的 95% 的改动，都不会碰到你的账本。**

2.2 为什么"开发模式"天然安全

回顾 1-2 讲：**程序目录和数据目录是分开的。**

```
程序目录：C:\Users\34123\AppData\Local\Programs\daily-ledger\
数据目录：<你的数据目录>\daily-ledger.db        ← 你的账
```

你改的是程序（`src\` 里的代码）。运行 `npm run dev` 时：

- 程序是**临时的**（跑在内存里，关了就没）
- 数据目录**还是那个** `<你的数据目录>`

所以：**改界面代码，你的账本完全不参与。**

2.3 唯一需要小心的情况

只有一种情况会真的动你的数据：

1. 你写了一个**新的迁移文件**（2-3 讲那种），并且
2. 你用**真实数据目录**启动了程序

这时迁移会真的执行。

防护措施：**做迁移实验时，永远用临时目录。**项目里所有验证脚本都是这么做的——下一节详细讲。

三、项目的验证体系

3.1 你有 15 个"自动检查员"

这个项目有一整套自动化验证。**先看它们都是什么**：

```
cd "C:\Users\34123\Desktop\记账簿"
node -e "const s=require('./package.json').scripts; Object.keys(s).filter(k=>/^(verify|sel
```

实际有 15 个。逐个说明它们防什么：

命令	防的问题	耗时
`npm run typecheck`	**类型错误**：字段拼错、参数用错	几秒
`npm run selftest`	**数据层逻辑错**：**88 项检查**（增删改查、余额口径、迁移……）	**8 秒**
`npm run verify:upgrade`	**升级事故**：老用户的库能不能安全升级	十几秒
`npm run smoke:pages`	**白屏**：每个页面能不能正常渲染	约 1 分钟
`npm run verify:motion`	**动效被破坏**：动画还在不在	约 1 分钟
`npm run verify:billshot`	**截图识别解析错**	几秒
`npm run verify:exchange`	**导入导出往返错**	几秒
`npm run verify:security`	**应用锁与备份逻辑错**	几秒
`npm run verify:update`	**检查更新的网络分支**	几秒
`npm run verify:ocr`	**截图识别引擎（需要模型文件）**	视图片而定
`npm run verify:site`	**官网 SEO 元素缺失**	1 秒
`npm run verify:packaged`	**打包产物坏**：托盘、写入、重启读回	约 1 分钟

注意 `selftest` 的规模：88 项检查，只花 8 秒。

这就是自动化验证的价值：8 秒就能确认"数据层逻辑没坏"。 换成人工点 88 次，得半小时，而且必然漏。

3.2 它们为什么不碰你的数据（关键机制）

这是本讲最有价值的一节。 打开 `scripts\run-probe.mjs`，看这几行（第 26–32 行）：

```
const sandbox = path.join(root, '.tmp', probeEnv.toLowerCase())
const userData = path.join(sandbox, 'userdata')
const dataDir = path.join(sandbox, 'data')
rmSync(sandbox, { recursive: true, force: true })
mkdirSync(userData, { recursive: true })
mkdirSync(dataDir, { recursive: true })
writeFileSync(path.join(userData, 'config.json'),
  JSON.stringify({ initialized: true, dataDir }, null, 2), 'utf-8')
```

逐行翻译：

代码	意思
`.tmp\...`	在项目的 `.tmp\` 目录下建一个**沙箱**
`rmSync(sandbox, ...)`	**先删掉旧的沙箱**（每次都从干净状态开始）
`mkdirSync(userData)`	建一个**假的**"用户配置目录"
`mkdirSync(dataDir)`	建一个**假的**"数据目录"
`writeFileSync(...config.json...)`	**写一份假的配置文件**，把数据目录指向沙箱

然后启动程序时用 `--user-data-dir=<沙箱>`，程序就以为"我的配置和数据都在这里"，完全不知道 `<你的数据目录>` 的存在。

这就是"沙箱隔离"：

```
真实环境：config.json → <你的数据目录>\daily-ledger.db        ← 你的账（验证脚本不碰）
沙箱环境：config.json → 项目\.tmp\xxx\data\daily-ledger.db  ← 一次性的假账
```

这个机制值得你学会，因为：

1. **它让你敢做实验**——知道最坏情况也就是删掉一个临时文件夹
2. **它是个通用思路**：任何"有副作用的操作"，都应该在隔离环境里先试

我在第 2-3 讲给过你实测证据：跑完 `verify:upgrade` 后，你的数据库**字节数和修改时间都没变**。就是这个机制在起作用。

3.3 怎么读验证结果

验证脚本的输出格式是统一的：`PASS` / `FAIL` + **一句说明。**

```
PASS  余额口径正确  —  收入 12550 − 支出 3069.8 = 结余 9480.2
FAIL  迁移后字段缺失  —  期望 tags 字段，实际没有
===== 数据层自检：88/89 通过 =====
```

怎么用：

情况	说明	该做什么
全 PASS	一切正常	继续下一步
有 FAIL	有东西坏了	**先看 FAIL 的那一条**，别管总数
报错退出	脚本自己崩了	看报错信息，通常是环境问题（比如没构建）

一个实用技巧：验证脚本的输出很长，**只看 FAIL 那几行**：

```
npm run selftest 2>&1 | Select-String "FAIL"
```

输出为空 = 全通过。

四、没有版本控制怎么办（本项目的真实约束）

4.1 先说个坏消息

好消息：这个项目现在已经接入 git 了（远程仓库在 GitHub）。但你要知道——**它是学完这套讲义之后才加上的。** 最开始做记账簿时，项目里确实没有 `.git`，所有改动只能靠手工备份。下面先讲那种情况，因为它是最坏的情况，也是最容易踩坑的。

git 是什么：一个"代码版本管理"工具。有了它，你可以：

- 随时回到任何一个历史版本（`git checkout`）
- 看到"我改了哪些行"（`git diff`）
- 放心大胆地改——反正能退回去

没有 git 意味着：**改坏了没法一键还原**，只能靠手工备份和手工撤销（3-4 讲那种"自己删掉加的两行"，很容易漏）。

4.2 三个替代方案（按推荐顺序）

方案 A：动手前复制备份（最简单，最常用）

```
# 改之前，把要动的文件复制一份到临时目录
Copy-Item src\renderer\src\features\records\RecordTable.tsx "$env:TEMP\RecordTable.backup.
```

改坏了怎么办：

```
Copy-Item "$env:TEMP\RecordTable.backup.tsx" src\renderer\src\features\records\RecordTable
```

为什么放 `$env:TEMP`：那是系统的临时目录，不在项目里，**不会污染项目。**

方案 B：整目录备份（改动较大时）

```
# 把整个 src 复制一份（几 MB，很快）
Copy-Item src "$env:TEMP\src-backup-$(Get-Date -Format 'MMdd-HHmm')" -Recurse
```

方案 C：用 git（现在这个项目已经接入了，直接用这个）

先讲清楚：git 本质上是"给代码拍的快照"。每次 `commit` 就是拍一张照，以后任何时候都能回到任何一张。

这个项目现在的 git 状态：

项	值
远程仓库	`https://github.com/karina2256/simple-ledger`
分支	`master`
已有提交	2 个（初始提交 + 文档更新）

你只需要记 6 条命令（下面每条都用记账簿做例子）：

```
cd "C:\Users\34123\Desktop\记账簿"

# ① 看现在什么状态（哪些文件改了）
git status

# ② 看具体改了哪几行（最有用的一条）
git diff

# ③ 改坏了，把某个文件还原到上次提交
git restore src/renderer/src/features/records/RecordTable.tsx

# ④ 改好了，保存成一个快照
git add -A
git commit -m "给记录列表加了净额列"

# ⑤ 推到 GitHub（备份到远程）
git push

# ⑥ 看历史（自己走过哪些路）
git log --oneline
```

下面这张对照表，就是"有 git"比"手工备份"强在哪：

场景	没有 git	有 git
想知道自己改了什么	想不起来	`git diff` 一眼看到
改坏了某一个文件	靠备份恢复，可能备份也忘了	`git restore 文件名`
改坏了想全部退回	手工逐个撤销，必然漏	`git restore .`
想回到三天前的状态	做不到	`git log` 找到那个提交，`git checkout`
电脑坏了代码没了	全没了	**GitHub 上还有一份**

最后一行是重点：git 不只是"后悔药"，还是"异地备份"。 你的代码现在同时存在于你的电脑和 GitHub 上，**硬盘坏了也不会丢。**

建议的日常节奏：

```
写完一个小功能 / 修完一个 bug  →  git add -A && git commit -m "..."
一天结束  →  git push
```

别攒太久：攒一周再提交，中间出问题就没法精细回退了。**"能跑通就提交一次"是个好习惯。**

> **注意**：`git restore` 和 `git checkout` 都是从 **git 里存的快照**恢复，**不会**动你的记账数据（`<你的数据目录>` 那个数据库在 git 之外，从来没被 git 管过）。这一点可以放心。

4.3 一个真实的教训

我在给你做这套讲义的过程中，真的踩过一次"没有版本控制"的坑：

在做官网时，我用一个命令提取打包文件里的 `package.json`，结果**它把项目根目录的 `package.json` 覆盖掉了**——项目的"身份证"被我弄没了。

怎么恢复的：靠 `package-lock.json`（它里面记录了所有依赖的版本）**反推出原始内容**，手工重建。

这次教训说明两件事：

1. **危险命令要先想清楚它会不会覆写文件**（我那条命令的本意只是"读出来看看"）
2. **有备份/有 git 就不会这么狼狈**

我在讲义里讲这个，是因为它真实发生过，而且以后也可能发生在你身上。

五、动手任务

步骤 1：确认基线（养成习惯）

```
cd "C:\Users\34123\Desktop\记账簿"
npm run typecheck
```

预期：**没有任何输出**（0 错误）。

这句话的意思是："现在项目是干净的，接下来如果出错，一定是我改出来的。"

这就是"基线"。每次动手前都做这一步，只花几秒。

如果没有输出让你不放心，用这条：

```
npm run typecheck 2>&1 | Select-String "error TS" | Measure-Object | Select-Object -Expand
```

预期输出： `0` **。**

步骤 2：跑一次数据层自检（看看 88 项检查长什么样）

```
npm run selftest 2>&1 | Select-Object -Last 12
```

预期看到类似：

```
PASS  新建账户后余额正确  -  ...
PASS  软删除后查询不到  -  ...
===== 数据层自检：88/88 通过 =====
```

花 8 秒，确认 88 件事。这就是自动化验证的性价比。

再看看它检查了什么：

```
npm run selftest 2>&1 | Select-String "PASS" | Select-Object -First 20
```

你会发现：这些检查覆盖了余额口径、软删除、迁移、附件策略、日历汇总、周期规则……**基本涵盖了所有数据层的关键行为。**

步骤 3：验证"沙箱隔离"是真的（重要）

任务：跑一个验证脚本，同时**观察你的真实数据有没有被碰。**

```
# ① 记录当前状态
(Get-Item "<你的数据目录>\daily-ledger.db").Length
(Get-Item "<你的数据目录>\daily-ledger.db").LastWriteTime

# ② 跑一个会创建大量数据的验证（CSV 导入导出往返）
npm run verify:exchange 2>&1 | Select-String "验证："

# ③ 再看一次
(Get-Item "<你的数据目录>\daily-ledger.db").Length
(Get-Item "<你的数据目录>\daily-ledger.db").LastWriteTime
```

预期：**前后两次完全一样**（字节数和修改时间都没变）。

再看沙箱目录里有什么：

```
Get-ChildItem .tmp -Directory | Select-Object Name
Get-ChildItem .tmp\ledger_exchange_probe\data -ErrorAction SilentlyContinue
```

你会看到：验证脚本在 `.tmp\` 下建了自己的数据库——**它一直在用假数据工作。**

这一课的意义：**你现在有底气说"跑验证不会弄坏我的账"**，因为你亲眼验证过。**而不是因为我告诉你是这样。**

步骤 4：练习"备份 → 改 → 验证 → 还原"完整循环

做一次最小实验，体验完整流程。

① 备份：

```
Copy-Item src\renderer\src\features\records\RecordTable.tsx "$env:TEMP\RecordTable.backup.
```

② 改一点点（用 VS Code 打开这个文件，把任意一个中文列名改掉，比如把「备注」改成「备注备注」）。

③ 验证：

```
npm run typecheck
```

预期：**0 错误**（改文字不会造成类型错误）。

④ 还原：

```
Copy-Item "$env:TEMP\RecordTable.backup.tsx" src\renderer\src\features\records\RecordTable
```

⑤ 确认还原干净：

```
Select-String -Path src\renderer\src\features\records\RecordTable.tsx -Pattern "备注"
```

预期：看到正常的「备注」，没有「备注备注」。

这就是一个完整的"安全改动循环"。 以后你每次改代码，都走这五步。

步骤 5：认识"全套回归"（模块 5 会用到）

如果你改的东西比较大，需要跑的不止一个命令。项目里有一套"全套回归"：

```
# 依次跑这些（每个都要通过）
npm run typecheck
npm run selftest
npm run smoke:pages
npm run verify:motion
```

为什么是这几个：它们分别覆盖**类型安全、数据逻辑、页面渲染、动效——四类最容易被改坏的东西。**

注意耗时：全套大约 **2–3 分钟。不算快，但比"上线后才发现坏了"便宜得多。**

卡住了怎么办

现象	原因	怎么办
`typecheck` 有报错	项目当前不干净（或你之前改的）	**先修掉再动手**，别在脏基线上改
`selftest` 报错找不到模块	没有构建过	先跑 `npm run build`
验证脚本超时	机器忙 / 首次构建慢	重跑一次；或先单独 `npm run build`
忘了备份就改了	常见	看 `.tmp` 里有没有历史产物；或手工把改动改回去
不确定某个文件能不能动	谨慎是对的	**先备份再动**，或先只读不写

六、自测题

1. 改代码的六步流程是什么？每一步在防什么？

2. 为什么动手前要先跑一次 `typecheck` 确认"基线干净"？

3. 你的改动什么时候会真的碰到账本数据？

4. 验证脚本为什么不碰你的真实数据？

5. 有 git 和没 git 时，"改坏了怎么还原"分别怎么做？

答案与解析

1. ① **确认基线**（防：把原有问题当成自己改的）② **分析影响**（防：漏改）③ **备份**（防：无法还原）④ **最小改动**（防：定位不到问题）⑤ **验证**（防：破坏别的功能）⑥ **收尾**（防：留下垃圾文件）。

2. 因为改完之后如果出现报错，**你需要知道这报错是不是你造成的。** → 基线干净 = "接下来任何错误都是我的责任"，**排查范围立刻缩小到你的改动。**这是最高效的排查起点。

3. **只有一种情况**：你写了新的**数据库迁移**，并且用**真实数据目录**启动了程序。 → 改界面（文字、颜色、布局、加显示列）**完全不会**碰数据；改主进程逻辑也**不会动已有数据**（只影响新记录的行为）。

4. 因为验证脚本会在项目的 `.tmp\` 下建一个**沙箱**：假的配置目录 + 假的数据目录，然后用 `--user-data-dir` 启动程序。 → 程序以为"我的数据在这里"，**根本不知道 `<你的数据目录>` 的存在**。你可以自己验证：跑验证前后对比数据库的字节数与修改时间，应该完全一样。

5. 没有 git：手工备份 + 还原（`Copy-Item` 到临时目录）——改动较大时备份整个 `src`。 **有 git**（本项目现在就有）：`git diff` 看改了什么、`git restore 文件名` 一键还原、`git log` 看历史、`git push` 备份到 GitHub。 → **git 不只是后悔药，还是异地备份**——代码同时在你的电脑和 GitHub 上，硬盘坏了也不丢。

七、术语表

术语	一句话解释
基线 / Baseline	动手前的"干净状态"，用来区分新旧问题
回归测试 / Regression	确认"原有功能没被新改动破坏"的测试
沙箱 / Sandbox	隔离的试验环境，坏了也不影响真实数据
自动化验证	用脚本代替人工检查，快且不会漏
影响面分析	动手前先搞清"要改哪些文件、会波及谁"
最小改动	只改必须改的，不顺手做别的
版本控制 / git	记录代码历史、随时回退的工具（本项目未使用）
备份 / Backup	改动前的文件副本，用于还原
PASS / FAIL	验证脚本的两种结果
`.tmp\`	项目的临时目录，验证脚本的沙箱都建在这里

八、本讲与项目的对应

- 你执行的命令：`typecheck`、`selftest`、`verify:exchange`（全部安全）

- 你做的实验：备份 → 改文字 → 类型检查 → 还原
- **本讲的所有操作都不会影响你的记账数据**（你亲眼验证过沙箱隔离）
- 上一讲：3-4 动手加一列 ｜ 下一讲：**4-2 从需求到设计**（完整做一个真实小功能的上半场）

讲义版本：v1 ｜ 对应项目版本：0.1.0